

Compiler Design Lab

Assignment – 1

by

Avyakth Shriram (24BCE5461)

1) Objective:

- **Lexical Tokenization:** Design a deterministic Lexical Analyzer using FLEX to categorize raw C character inputs into clean token expressions.
- **Symbol Table Architecture:** Construct an advanced, multi-attribute, 8-column structured Symbol Table to catalogue identifiers, variable attributes, storage tracking metrics, and block nest hierarchies.
- **Algorithmic Duality:** Create two structurally complete solutions for evaluation:
 1. A unified, integrated FLEX tracking scanner pipeline.
 2. A raw, standalone line-buffer lookup compiler automation program written purely in ANSI C.
- **Strict Parameter Integrity:** Resolve structural property mappings solely within sequential token string processors without utilizing explicit language parser generators like BISON or YACC.

2) Theoretical Overview & Compilation Parameters:

A lexical token processor breaks source arrays into foundational semantic structures called **Tokens**. During this extraction loop, declaration identifiers are recorded inside a **Symbol Table** object cache.

To maintain complex language variables without a formal parser tree engine, this implementation relies on custom global variable state machines:

- **Scope Tracking:** Tracks curly brace block boundaries ({ and }). A tracking counter state registers parent-child boundaries, recording structural depth names dynamically (e.g., Global, Local (Lvl 1), Local (Lvl 2)).
- **Context Type Allocation:** A structural buffer string variable caches the most recently evaluated keyword type structure (e.g., int, float, double). This context tag assigns storage structures to newly detected variable arrays and resets cleanly upon statement boundaries (;).
- **Memory Footprint Calculations:** Simulates execution data alignments by initializing sequential pointer lookups starting from a base hex location (0x3E8). Every confirmed identifier scales the global address tracking pointer ahead relative to its standard system architecture data type size rules:

int = 4 bytes, float = 4 bytes, char = 1 byte, double = 8 bytes

3) Core System Architecture:

3.1) Resolving Ambiguities with Trailing Context (/)

In FLEX regular expressions, if two distinct token validation options match text of identical lengths, the rule written higher in the file triggers. However, if one pattern matches a longer segment of characters, **the longest match always wins**.

This property introduces a parsing bug when using a basic lookahead signature like `[a-zA-Z_][a-zA-Z0-9_]*[ \t]*\((`. When processing code containing conditional keywords like `if (a > 5)`, the keyword rule matches only 2 characters (`if`), while the function rule matches 4 characters (`if ()`). As a result, the lexer misclassifies `if` as a function identifier.

To resolve this ambiguity, this architecture uses the FLEX **trailing context operator (/)**:

`[a-zA-Z_][a-zA-Z0-9_]*/[ \t]*\((`

This syntax looks ahead to check if the identifier is followed by spaces and a parenthesis, but **only consumes the identifier token itself**. This ensures that conditional keywords like `if` and function calls like `main return` matching strings of equal lengths, allowing the higher-priority keyword rule to resolve correctly.

3.2) Matrix Data Expansion Scaling

When the scanner intercepts a dimension structural bound indicator ([5]), it locates the preceding element inside the sequential structure cache array. It modifies the baseline storage allocation using an arithmetic calculation block:

$$\text{Adjusted Footprint Size} = \text{Baseline Architecture Size} * \text{Extracted Array Dimension Dimension Bound}$$

The global memory allocation tracking offset counter is then updated to reflect this calculated data structure alignment.

4) Standalone C-language Symbol Table Generator: (symbol_table_generator.c)

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>

#define MAX_SYMBOLS 100

// --- Symbol Table Structural Definition ---
struct Symbol {
    int id;
    char name[50];
    char type[20];
    char kind[20];
    char scope[30];
    int address;
    int line_no;
    int size;
    char dimensions[20];
};

struct Symbol symTable[MAX_SYMBOLS];
int symCount = 0;

// Tracker state variables
int currentScopeLevel = 0;    // 0 = Global, 1 = Local/Function, 2+ = Nested blocks
int currentOffset = 1000;    // Simulated base memory address starting at 1000

// Helper to calculate data type size
int getDataTypeSize(char *type) {
    if (strcmp(type, "int") == 0) return 4;
    if (strcmp(type, "float") == 0) return 4;
    if (strcmp(type, "char") == 0) return 1;
```

```

    if (strcmp(type, "double") == 0) return 8;
    return 0;
}

// Function to add/update identifier in Symbol Table
void insertSymbol(char *name, char *type, char *kind, int scopeLvl, int line, int size, char *dims) {
    char scopeStr[30];
    if (scopeLvl == 0) strcpy(scopeStr, "Global");
    else sprintf(scopeStr, "Local (Lvl %d)", scopeLvl);

    // Check if symbol already exists in the current scope
    for(int i = 0; i < symCount; i++) {
        if(strcmp(symTable[i].name, name) == 0 && strcmp(symTable[i].scope, scopeStr) == 0) {
            return;
        }
    }

    if (symCount >= MAX_SYMBOLS) return;

    symTable[symCount].id = symCount + 1;
    strcpy(symTable[symCount].name, name);
    strcpy(symTable[symCount].type, (strlen(type) > 0) ? type : "N/A");
    strcpy(symTable[symCount].kind, kind);
    strcpy(symTable[symCount].scope, scopeStr);

    symTable[symCount].address = currentOffset;
    symTable[symCount].line_no = line;
    symTable[symCount].size = size;
    strcpy(symTable[symCount].dimensions, (strlen(dims) > 0) ? dims : "None");

    currentOffset += size;
    symCount++;
}

// Helper to check for C type keywords
int isTypeKeyword(char *word) {
    return (strcmp(word, "int") == 0 || strcmp(word, "float") == 0 ||
            strcmp(word, "char") == 0 || strcmp(word, "double") == 0 ||
            strcmp(word, "void") == 0);
}

```

```
// Helper to check for keywords that can be followed by a '(' but are NOT functions
```

```
int isControlKeyword(char *word) {  
    return (strcmp(word, "if") == 0 || strcmp(word, "while") == 0 ||  
            strcmp(word, "for") == 0 || strcmp(word, "return") == 0);  
}
```

```
void generateSymbolTable(FILE *fp) {
```

```
    char line[256];  
    int lineNum = 0;  
    char currentType[20] = "";  
    int inMultiLineComment = 0;
```

```
    while (fgets(line, sizeof(line), fp)) {  
        lineNum++;  
        int len = strlen(line);  
        int i = 0;
```

```
        while (i < len) {  
            // Handle multi-line comment exit  
            if (inMultiLineComment) {  
                if (line[i] == '*' && i + 1 < len && line[i + 1] == '/') {  
                    inMultiLineComment = 0;  
                    i += 2;  
                } else {  
                    i++;  
                }  
                continue;  
            }  
            if (line[i] == '/' && i + 1 < len && line[i + 1] == '*') {  
                inMultiLineComment = 1;  
                i += 2;  
                continue;  
            }  
            i++;  
        }
```

```
        // Skip spaces/tabs  
        if (isspace(line[i])) {  
            i++;  
            continue;  
        }
```

```
        // Handle Single-line and Multi-line comment entries  
        if (line[i] == '/' && i + 1 < len) {  
            if (line[i + 1] == '/') break; // Skip the rest of the line  
            if (line[i + 1] == '*') {  
                inMultiLineComment = 1;  
                i += 2;  
                continue;  
            }  
            i++;  
        }
```

```

        inMultiLineComment = 1;
        i += 2;
        continue;
    }
}

// Track Scopes
if (line[i] == '{') {
    currentScopeLevel++;
    i++;
    continue;
}
if (line[i] == '}') {
    if (currentScopeLevel > 0) currentScopeLevel--;
    i++;
    continue;
}
if (line[i] == ';') {
    strcpy(currentType, ""); // Reset active declaration token type context
    i++;
    continue;
}

// Extract alphanumeric words
if (isalpha(line[i]) || line[i] == '_') {
    char buffer[50];
    int bufIdx = 0;
    while (i < len && (isalnum(line[i]) || line[i] == '_')) {
        if (bufIdx < 49) buffer[bufIdx++] = line[i];
        i++;
    }
    buffer[bufIdx] = '\0';

    if (isTypeKeyword(buffer)) {
        strcpy(currentType, buffer);
    }
    else if (strlen(currentType) > 0 && !isControlKeyword(buffer)) {
        char kind[20] = "variable";
        char dims[20] = "None";
        int allocatedSize = getDataTypeSize(currentType);
    }
}

```

```

// Peek explicitly ahead in the remaining characters on this line
int peek = i;
while (peek < len && isspace(line[peek])) {
    peek++;
}

if (peek < len && line[peek] == '(') {
    strcpy(kind, "function");
    allocatedSize = 4; // Function base pointer footprint
}
else if (peek < len && line[peek] == '[') {
    int dIdx = 0;
    while (peek < len && line[peek] != ']') {
        if (dIdx < 19) dims[dIdx++] = line[peek];
        peek++;
    }
    if (peek < len && line[peek] == ']') {
        dims[dIdx++] = ']';
    }
    dims[dIdx] = '\0';

    // Extract array size integer bound multiplier
    int bound = 0;
    for (int k = 0; dims[k] != '\0'; k++) {
        if (isdigit(dims[k])) {
            bound = bound * 10 + (dims[k] - '0');
        }
    }
    if (bound > 0) {
        allocatedSize *= bound;
    }
}

insertSymbol(buffer, currentType, kind, currentScopeLevel, lineNum, allocatedSize, dims);
}
} else {
    i++; // Fallback increment for operator/separator symbols
}
}

```

```

    }
}

int main(int argc, char **argv) {
    if (argc < 2) {
        printf("Usage: %s <source_file.c>\n", argv[0]);
        return 1;
    }

    FILE *fp = fopen(argv[1], "r");
    if (!fp) {
        perror("Error opening file");
        return 1;
    }

    generateSymbolTable(fp);
    fclose(fp);

    printf("\n=== Q1: COMPLETE SYMBOL TABLE GENERATION (STANDALONE C
PROGRAM) ===\n");
    printf("-----\n");
    printf("%-4s | %-12s | %-10s | %-10s | %-15s | %-10s | %-8s | %-6s | %-10s\n",
        "ID", "NAME", "TYPE", "KIND", "SCOPE/LEVEL", "MEM ADDR", "LINE NO", "SIZE",
"DIMENSIONS");
    printf("-----\n");
    for(int i = 0; i < symCount; i++) {
        printf("%-4d | %-12s | %-10s | %-10s | %-15s | 0x%-8X | %-8d | %-6d | %-10s\n",
            symTable[i].id,
            symTable[i].name,
            symTable[i].type,
            symTable[i].kind,
            symTable[i].scope,
            symTable[i].address,
            symTable[i].line_no,
            symTable[i].size,
            symTable[i].dimensions);
    }
}

```

```
printf("-----\n");  
  
return 0;  
}
```

5) Integrated Lexical Analyzer & Symbol Table: (lexer.l)

```
%{
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX_SYMBOLS 100

// --- Symbol Table Structural Definition ---
struct Symbol {
    int id;
    char name[50];
    char type[20];
    char kind[20];
    char scope[30];
    int address;
    int line_no;
    int size;
    char dimensions[20];
};

struct Symbol symTable[MAX_SYMBOLS];
int symCount = 0;

// Tracker state variables
char currentType[20] = "";
char currentKind[20] = "variable";
int currentScopeLevel = 0;    // 0 = Global, 1 = Local/Function, 2+ = Nested blocks
int currentOffset = 1000;    // Simulated base memory address starting at 1000
int lineNum = 1;             // Track current line number

// Helper to calculate data type size
int getDataTypeSize(char *type) {
```

```

    if (strcmp(type, "int") == 0) return 4;
    if (strcmp(type, "float") == 0) return 4;
    if (strcmp(type, "char") == 0) return 1;
    if (strcmp(type, "double") == 0) return 8;
    return 0;
}

// Function to add/update identifier in Symbol Table
void insertSymbol(char *name, char *type, char *kind, int scopeLvl, int line, int size, char *dims) {
    char scopeStr[30];
    if (scopeLvl == 0) strcpy(scopeStr, "Global");
    else sprintf(scopeStr, "Local (Lvl %d)", scopeLvl);

    // Check if symbol already exists in the current scope
    for(int i = 0; i < symCount; i++) {
        if(strcmp(symTable[i].name, name) == 0 && strcmp(symTable[i].scope, scopeStr) == 0) {
            return;
        }
    }

    if (symCount >= MAX_SYMBOLS) return;

    symTable[symCount].id = symCount + 1;
    strcpy(symTable[symCount].name, name);
    strcpy(symTable[symCount].type, (strlen(type) > 0) ? type : "N/A");
    strcpy(symTable[symCount].kind, kind);
    strcpy(symTable[symCount].scope, scopeStr);

    symTable[symCount].address = currentOffset;
    symTable[symCount].line_no = line;
    symTable[symCount].size = size;
    strcpy(symTable[symCount].dimensions, (strlen(dims) > 0) ? dims : "None");

    currentOffset += size;
    symCount++;
}

/* Rules Section */
%%

```

```

\n                { lineNum++; }
[ \t\r]+         { /* Ignore spaces and tabs */ }

"/".*            { printf("%-20s : %s\n", "COMMENT", yytext); }
"/*"([^\*]|\"+[^*/])*\*+ "/" { printf("%-20s : %s\n", "COMMENT", yytext); }

/* Keywords - Explicitly matched first */
"int"|"float"|"char"|"double"|"void" {
    printf("%-20s : %s\n", "KEYWORD (TYPE)", yytext);
    strcpy(currentType, yytext);
}

"return"|"if"|"else"|"while"|"for"|"do"|"break"|"continue" {
    printf("%-20s : %s\n", "KEYWORD", yytext);
}

/* MATCH FUNCTIONS ONLY using trailing context '/' */
/* This matches an identifier only if it's followed by optional spaces and a '(' */
/* It leaves the '(' in the stream to be matched by the separator rule later! */
[a-zA-Z_][a-zA-Z0-9_]*/[ \t]*\({
    // Exclude conditional keywords from being treated as functions
    if (strcmp(yytext, "if") == 0 || strcmp(yytext, "while") == 0 ||
        strcmp(yytext, "for") == 0 || strcmp(yytext, "return") == 0) {
        printf("%-20s : %s\n", "KEYWORD", yytext);
    } else {
        printf("%-20s : %s\n", "IDENTIFIER (FUNC)", yytext);
        if (strcmp(currentType, "") != 0) {
            insertSymbol(yytext, currentType, "function", currentScopeLevel, lineNum, 4, "None");
        }
    }
}

/* MATCH STANDARD VARIABLES */
[a-zA-Z_][a-zA-Z0-9_]* {
    printf("%-20s : %s\n", "IDENTIFIER", yytext);

    if (strcmp(currentType, "") != 0) {
        int size = getDataTypeSize(currentType);
        insertSymbol(yytext, currentType, "variable", currentScopeLevel, lineNum, size, "");
    }
}

```

```

    }
}

"["[0-9]+" {
    printf("%-20s : %s\n", "ARRAY DIMENSION", yytext);
    if (symCount > 0) {
        strcpy(symTable[symCount-1].dimensions, yytext);
        int bound = atoi(yytext + 1);
        symTable[symCount-1].size *= bound;
        currentOffset += (symTable[symCount-1].size - symTable[symCount-1].size/bound);
    }
}

[0-9]+      { printf("%-20s : %s\n", "INTEGER CONSTANT", yytext); }
[0-9]*\.[0-9]+ { printf("%-20s : %s\n", "FLOAT CONSTANT", yytext); }
"\".\"\"\"    { printf("%-20s : %s\n", "CHAR CONSTANT", yytext); }

"=|"+|"-|"*|"/|"%|">|"<|">=|"<=|"==|"!==" {
    printf("%-20s : %s\n", "OPERATOR", yytext);
}

"{" {
    printf("%-20s : %s\n", "SEPARATOR", yytext);
    currentScopeLevel++;
}

"}" {
    printf("%-20s : %s\n", "SEPARATOR", yytext);
    if(currentScopeLevel > 0) currentScopeLevel--;
}

"("|")|"," {
    printf("%-20s : %s\n", "SEPARATOR", yytext);
}

";" {
    printf("%-20s : %s\n", "SEPARATOR", yytext);
    strcpy(currentType, "");
}

```

```

. { printf("%-20s : %s\n", "UNKNOWN", yytext); }

%%

int yywrap() { return 1; }

int main(int argc, char **argv) {
    if (argc > 1) {
        FILE *file = fopen(argv[1], "r");
        if (!file) {
            perror("Error opening file");
            return 1;
        }
        yyin = file;
    }

    printf("\n=== Q2: LEXICAL ANALYSIS OUTPUT ===\n");
    printf("%-20s  %s\n", "TOKEN TYPE", "VALUE");
    printf("-----\n");

    yylex();

    printf("\n\n=== Q1: COMPLETE SYMBOL TABLE GENERATION ===\n");
    printf("-----\n");
    printf("%-4s | %-12s | %-10s | %-10s | %-15s | %-10s | %-8s | %-6s | %-10s\n",
           "ID", "NAME", "TYPE", "KIND", "SCOPE/LEVEL", "MEM ADDR", "LINE NO", "SIZE",
           "DIMENSIONS");
    printf("-----\n");
    for(int i = 0; i < symCount; i++) {
        printf("%-4d | %-12s | %-10s | %-10s | %-15s | 0x%-8X | %-8d | %-6d | %-10s\n",
               symTable[i].id,
               symTable[i].name,
               symTable[i].type,
               symTable[i].kind,
               symTable[i].scope,
               symTable[i].address,
               symTable[i].line_no,
               symTable[i].size,

```

```
        symTable[i].dimensions);  
    }  
    printf("-----\n");  
    return 0;  
}
```

6) Verification Test Driver Framework: (input.c)

The following complex code structure tests the parser logic across global declarations, matrix transformations, local variables, conditional controls, and nested blocks:

```
// Complex Test Case
int globalVar;

int main() {
    int a = 10;
    float b[5];
    char choice;

    if (a > 5) {
        double nestedVal = 99.99;
    }
    return 0;
}
```

7) Output Verification & Screenshots:

symbol_table_generator.c

```
scope1@AB3-302-Sys56: ~/AvyakthS$ tree
.
├── Compiler-Design-Lab
│   ├── LAB1
│   │   ├── input.c
│   │   ├── lex.yy.c
│   │   ├── lexer.l
│   │   ├── symbol-table-generator_plus_lexer
│   │   ├── symbol_table_generator
│   │   └── symbol_table_generator.c
│   └── LAB2
└── 4 directories, 6 files

scope1@AB3-302-Sys56:~/AvyakthS$ cd Compiler-Design-Lab/LAB1
scope1@AB3-302-Sys56:~/AvyakthS/Compiler-Design-Lab/LAB1$ ls -a
.  ..  input.c  lex.yy.c  lexer.l  symbol-table-generator_plus_lexer  symbol_table_generator  symbol_table_generator.c

scope1@AB3-302-Sys56:~/AvyakthS/Compiler-Design-Lab/LAB1$ ./symbol_table_generator input.c

=== Q1: COMPLETE SYMBOL TABLE GENERATION (STANDALONE C PROGRAM) ===
```

ID	NAME	TYPE	KIND	SCOPE/LEVEL	MEM ADDR	LINE NO	SIZE	DIMENSIONS
1	globalVar	int	variable	Global	0x3E8	2	4	None
2	main	int	function	Global	0x3EC	4	4	None
3	a	int	variable	Local (Lvl 1)	0x3F0	5	4	None
4	b	float	variable	Local (Lvl 1)	0x3F4	6	20	[5]
5	choice	char	variable	Local (Lvl 1)	0x408	7	1	None
6	nestedVal	double	variable	Local (Lvl 2)	0x409	10	8	None

```
scope1@AB3-302-Sys56:~/AvyakthS/Compiler-Design-Lab/LAB1$ |
```

symbol-table-generator-plus-lexer.c

```
scope1@AB3-302-Sys56: ~/Avyakth$ tree
.
├── Compiler-Design-Lab
│   ├── LAB1
│   │   ├── input.c
│   │   ├── lex.yy.c
│   │   ├── lexer.l
│   │   ├── symbol-table-generator-plus_lexer
│   │   ├── symbol_table_generator
│   │   └── symbol_table_generator.c
│   └── LAB2
└── 4 directories, 6 files

scope1@AB3-302-Sys56:~/Avyakth$ cd Compiler-Design-Lab/LAB1
scope1@AB3-302-Sys56:~/AvyakthS/Compiler-Design-Lab/LAB1$ ls -a
.  ..  input.c  lex.yy.c  lexer.l  symbol-table-generator-plus_lexer  symbol_table_generator  symbol_table_generator.c

scope1@AB3-302-Sys56:~/AvyakthS/Compiler-Design-Lab/LAB1$ ./symbol-table-generator-plus_lexer input.c

=== Q2: LEXICAL ANALYSIS OUTPUT ===
TOKEN TYPE      VALUE
-----
COMMENT         : // Complex Test Case
KEYWORD (TYPE)  : int
IDENTIFIER       : globalVar
SEPARATOR       : ;
KEYWORD (TYPE)  : int
IDENTIFIER (FUNC): main
SEPARATOR       : (
SEPARATOR       : )
SEPARATOR       : {
KEYWORD (TYPE)  : int
IDENTIFIER       : a
OPERATOR        : =
INTEGER CONSTANT: 10
SEPARATOR       : ;
KEYWORD (TYPE)  : float
IDENTIFIER       : b
ARRAY DIMENSION : [5]
SEPARATOR       : ;
KEYWORD (TYPE)  : char
IDENTIFIER       : choice
SEPARATOR       : ;
KEYWORD         : if
SEPARATOR       : (
IDENTIFIER       : a
OPERATOR        : >
INTEGER CONSTANT: 5
SEPARATOR       : )
SEPARATOR       : {
```

```
SEPARATOR       : ;
KEYWORD (TYPE)  : int
IDENTIFIER (FUNC): main
SEPARATOR       : (
SEPARATOR       : )
SEPARATOR       : {
KEYWORD (TYPE)  : int
IDENTIFIER       : a
OPERATOR        : =
INTEGER CONSTANT: 10
SEPARATOR       : ;
KEYWORD (TYPE)  : float
IDENTIFIER       : b
ARRAY DIMENSION : [5]
SEPARATOR       : ;
KEYWORD (TYPE)  : char
IDENTIFIER       : choice
SEPARATOR       : ;
KEYWORD         : if
SEPARATOR       : (
IDENTIFIER       : a
OPERATOR        : >
INTEGER CONSTANT: 5
SEPARATOR       : )
SEPARATOR       : {
KEYWORD (TYPE)  : double
IDENTIFIER       : nestedVal
OPERATOR        : =
FLOAT CONSTANT  : 99.99
SEPARATOR       : ;
SEPARATOR       : }
KEYWORD         : return
INTEGER CONSTANT: 0
SEPARATOR       : ;
SEPARATOR       : }
```

```
=== Q1: COMPLETE SYMBOL TABLE GENERATION ===
```

ID	NAME	TYPE	KIND	SCOPE/LEVEL	MEM ADDR	LINE NO	SIZE	DIMENSIONS
1	globalVar	int	variable	Global	0x3E8	2	4	None
2	main	int	function	Global	0x3EC	4	4	None
3	a	int	variable	Local (Lvl 1)	0x3F0	5	4	None
4	b	float	variable	Local (Lvl 1)	0x3F4	6	20	[5]
5	choice	char	variable	Local (Lvl 1)	0x408	7	1	None
6	nestedVal	double	variable	Local (Lvl 2)	0x409	10	8	None

```
scope1@AB3-302-Sys56:~/AvyakthS/Compiler-Design-Lab/LAB1$
```

8) Analytical Comparison Matrix:

The structural mechanics behind processing lookahead vectors and symbol bindings reveal distinct differences between the two engine implementations:

Attribute Category	Part A: FLEX Pipeline Architecture (lexer.l)	Part B: Standalone C Engine (symbol_table_generator.c)
Matching Engine	Uses Deterministic Finite Automata (DFA) state machines via regular expressions.	Uses a sequential loop to parse text lines character-by-character.
Lookahead Vector Rules	Uses Trailing Context (/) to match matching characters without consuming stream buffers.	Uses a lookahead index pointer index to scan remaining characters inside array frames.
Keyword Handling	Uses hardcoded regular expression priority layers.	Uses helper function string evaluations (strcmp) to confirm keywords.
Robustness	Native protection against buffering faults.	Requires exact bounds allocations to prevent memory leaks.

9) Conclusion:

Both compiler engines compile cleanly with **zero warnings** and output identical formatting tables. The logic correctly handles tracking configurations, such as computing system memory footprint expansions for matrix bounds (20 bytes for float b[5]), mapping local variables across nested execution hierarchies, and isolating global symbols from block declarations.

THANK YOU!